

Quick Sort

1. First Element as Pivot

CODE:

```
#include <stdio.h>

int partition(int a[], int low, int high) {
    int pivot = a[low];
    int i = low + 1;
    int j = high;
    int temp;

    while (i <= j) {
        while (i <= high && a[i] <= pivot)
            i++;
        while (a[j] > pivot)
            j--;
        if (i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    a[low] = a[j];
    a[j] = pivot;
    return j;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

```

int main() {
    int a[] = {5, 3, 8, 4, 2};
    int n = 5;

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

OUTPUT:

```

keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc quick_sort.c -o quick_sort
keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./quick_sort
2 3 4 5 8 %

```

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

2. Last Element as Pivot

CODE:

```
#include <stdio.h>

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    int temp;

    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;

    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

```

int main() {
    int a[] = {5, 3, 8, 4, 2};
    int n = 5;

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

OUTPUT:

```

keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc quick_sort.c -o quick_sort
keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./quick_sort
2 3 4 5 8

```

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

3. Random Element as Pivot

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    int temp;

    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;

    return i + 1;
}

int randomPartition(int a[], int low, int high) {
    int randomIndex = low + rand() % (high - low + 1);
    int temp = a[randomIndex];
    a[randomIndex] = a[high];
    a[high] = temp;

    return partition(a, low, high);
}
```

```

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = randomPartition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    int a[] = {5, 3, 8, 4, 2};
    int n = 5;

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

OUTPUT:

```

keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc quick_sort.c -o quick_sort
keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./quick_sort
2 3 4 5 8 %

```

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$